

# Life Orchestrator — Full Stack Report

**Author:** Ismail Abdullah (Ismailab1)

**Copyright:** © 2026 Ismail Abdullah (Ismailab1). All rights reserved.

**Purpose:** Authoritative reference of every technology layer in this project. Use this document to answer "what did you build with?" questions in applications, technical writeups, and portfolio presentations.

## 1. Application Overview

| Field        | Value                                                                                                                               |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Project name | Life Orchestrator                                                                                                                   |
| Type         | AI-powered life-management single-page application (SPA)                                                                            |
| Status       | Live on Google Cloud Run                                                                                                            |
| Live URL     | <a href="https://life-orchestrator-662200881058.us-central1.run.app">https://life-orchestrator-662200881058.us-central1.run.app</a> |
| Repository   | Ismailab1/Life-Orchestrator                                                                                                         |

**One-sentence summary:**

A local-first, conversational AI agent that orchestrates schedules, tracks relationship health, and manages a life inventory — all without a backend database.

## 2. Frontend

| Layer                  | Technology              | Version | Role                                                      |
|------------------------|-------------------------|---------|-----------------------------------------------------------|
| UI Framework           | React                   | 19.x    | Component tree, virtual DOM, state                        |
| Language               | TypeScript              | ~5.8    | Type safety across all source files                       |
| Styling                | Tailwind CSS            | 3.4.x   | Utility-first atomic CSS                                  |
| Markdown rendering     | react-markdown          | 9.0.1   | Renders AI-generated responses safely                     |
| Browser storage client | idb                     | 8.x     | IndexedDB wrapper (schema laid out, migration path ready) |
| Routing                | react-router-dom        | 7.x     | Client-side SPA routing                                   |
| Build tool             | Vite                    | 6.x     | ES module dev server, HMR, production bundler             |
| PostCSS                | autoprefixer + postcss  | latest  | Tailwind CSS pipeline                                     |
| Typography plugin      | @tailwindcss/typography | 0.5.x   | prose classes for AI markdown                             |

**Markup / markup-adjacent:**

- HTML5 ( `index.html` entry point)
- CSS3 (via Tailwind utility classes; no hand-written CSS files)

**Notable frontend patterns:**

- View state machine replaces React Router for navigation between app sections
- Inline editing pattern (click-to-edit, auto-save on blur) — no modal forms
- Optimistic UI — UI updates immediately, AI confirmation arrives asynchronously
- Streaming UX — AI responses render token-by-token via `ReadableStream`
- Multimodal input — text, Base64 images, and voice (microphone permission requested)

**3. AI / Machine Learning Integration**

| Component          | Technology                                                    | Notes                                                                 |
|--------------------|---------------------------------------------------------------|-----------------------------------------------------------------------|
| AI model           | Google Gemini 2.5 Pro                                         | Primary orchestration and reasoning model                             |
| SDK                | @google/generative-ai 0.24.x                                  | Official Google SDK for Gemini API                                    |
| Interaction mode   | Function calling (tool use)                                   | AI calls pre-defined TypeScript executor functions                    |
| Context strategy   | Prompt engineering as application logic                       | System prompt in <code>constants.ts</code> encodes all AI behavior    |
| Temporal modes     | Past / Today / Future                                         | AI switches behavior: Reflection / Active / Planning                  |
| Streaming          | <code>generateContentStream</code>                            | Token-by-token text delivery to UI                                    |
| Abort support      | <code>AbortController</code>                                  | Cancels in-flight Gemini streams on user request or date change       |
| Timeout protection | Custom <code>withTimeout()</code> wrapper                     | 30s stream, 30s chunk, 60s orchestration timeouts                     |
| Memory system      | Structured <code>Memory[]</code> in <code>localStorage</code> | AI reads persistent user preferences/facts across sessions            |
| Security boundary  | Executor function validation layer                            | AI cannot run arbitrary code; all actions route through typed schemas |

**Gemini tool functions implemented:**

- `save_day_plan` — persists structured schedule
- `update_relationship_status` — modifies Kinship Ledger entries
- `log_checkin` — records relationship contact event
- `save_memory` — stores long-term user facts
- `propose_orchestration` — emits a structured schedule proposal card in chat
- `get_life_context` — reads current inventory/ledger state into AI context
- `get_relationship_status` — queries specific relationship health

## 4. Backend / Server

| Component           | Technology                      | Notes                                                                 |
|---------------------|---------------------------------|-----------------------------------------------------------------------|
| Runtime             | Node.js (Alpine)                | Production container                                                  |
| HTTP framework      | Express.js                      | Minimal web server                                                    |
| API proxy           | http-proxy-middleware           | Proxies /api/gemini/* → Google Generative Language API                |
| Secret management   | Cloud Run environment variables | GEMINI_API_KEY never bundled into client JS                           |
| Static file serving | express.static                  | Serves the Vite-built dist/ SPA                                       |
| SPA fallback        | app.get('*') catch-all          | Returns index.html for client-side routes                             |
| CORS headers        | Custom setHeaders middleware    | Cross-Origin-Opener-Policy: same-origin-allow-popups for Google OAuth |

## 5. Data Layer

| Storage                 | Technology                       | What is stored                                         |
|-------------------------|----------------------------------|--------------------------------------------------------|
| Primary persistence     | Browser localStorage             | Tasks, messages, ledger, memories, orchestration state |
| Future migration target | Browser IndexedDB (via idb)      | Schema defined in services/db.ts; not yet activated    |
| External calendar       | Google Calendar API (GAPI + GSI) | Bi-directional event sync                              |

**No server-side database.** All user data is local to the browser. This is an intentional privacy-by-design decision.

**Storage quota management:**

- Hard limit: ~5 MB per origin
- Real-time percentage bar with amber (80%) and red (90%) warnings
- Granular cleanup by date; surgical delete rather than full reset
- Image compression pipeline: 800×800 px, JPEG 70% quality ≈ 100 KB
- Base64 overhead accounted for (×1.33 multiplier)

## 6. External APIs & Cloud Services

| Service                    | Provider     | Purpose                                    |
|----------------------------|--------------|--------------------------------------------|
| Gemini AI API              | Google Cloud | LLM inference, function calling, streaming |
| Google Calendar API (GAPI) | Google Cloud | Read/write calendar events                 |

|                                |              |                              |
|--------------------------------|--------------|------------------------------|
| Google Identity Services (GSI) | Google Cloud | OAuth 2.0 sign-in popup flow |
| Cloud Run                      | Google Cloud | Serverless container hosting |
| Container Registry             | Google Cloud | Docker image storage         |

## 7. Infrastructure & Deployment

| Component                      | Technology                              | Notes                                                           |
|--------------------------------|-----------------------------------------|-----------------------------------------------------------------|
| Containerization               | Docker (multi-stage)                    | Stage 1: Node 22 Alpine build; Stage 2: lean production image   |
| Container orchestration        | Google Cloud Run                        | Serverless, auto-scaling, zero cold-start cost                  |
| Deployment automation          | PowerShell script (advanced_deploy.ps1) | Local build → push → Cloud Run deploy                           |
| Reverse proxy / static serving | Express.js (built-in)                   | No nginx in final image                                         |
| Ignored build artifacts        | .gcloudignore                           | Excludes node_modules, .git, source files from Cloud submission |

### Docker build strategy:

```
Stage 1 (build) → Node 22 Alpine → npm ci → vite build → dist/  
Stage 2 (serve) → Node 22 Alpine → copy dist/ + server/ → npm install --omit=dev → node index.js
```

## 8. Developer Tooling

| Tool                   | Purpose                                      |
|------------------------|----------------------------------------------|
| TypeScript             | Compile-time type checking across all source |
| Vite                   | Dev server (HMR <50ms) + production bundler  |
| Tailwind CSS CLI       | CSS purge + JIT compilation                  |
| PostCSS + Autoprefixer | Cross-browser CSS compatibility              |
| .env + Vite define     | Build-time env variable injection (dev only) |

## 9. Key Source Files Reference

| File    | Role                                                                     |
|---------|--------------------------------------------------------------------------|
| App.tsx | Root component; state machine, all executor functions, AI stream handler |

|                                   |                                                                           |
|-----------------------------------|---------------------------------------------------------------------------|
| constants.ts                      | System prompt (SYSTEM_INSTRUCTION), temporal mode instructions            |
| types.ts                          | Full data model: Task, Person, Memory, ChatMessage, OrchestrationProposal |
| services/geminiService.ts         | Gemini SDK wrapper; chat session, streaming, tool definitions             |
| services/storageService.ts        | localStorage read/write helpers, quota management                         |
| services/googleCalendarService.ts | GAPI/GSI OAuth + calendar event CRUD                                      |
| services/imageService.ts          | Image compression pipeline                                                |
| services/db.ts                    | IndexedDB schema (future migration)                                       |
| components/ChatInterface.tsx      | Conversational UI, message stream rendering                               |
| components/KinshipLedger.tsx      | Relationship tracking UI                                                  |
| components/CareerInventory.tsx    | Task/inventory management UI                                              |
| components/LandingPage.tsx        | Marketing/onboarding page                                                 |
| server/index.js                   | Express proxy server (production)                                         |
| Dockerfile                        | Multi-stage container definition                                          |
| vite.config.ts                    | Build config, CORS headers, code splitting                                |
| tailwind.config.js                | Content paths, typography plugin                                          |

## 10. Lines of Code Snapshot (approximate)

| Area                             | Approx. LOC    |
|----------------------------------|----------------|
| App.tsx (root logic)             | ~1,200+        |
| services/                        | ~700+          |
| components/ (9 files)            | ~1,500+        |
| types.ts + constants.ts          | ~400+          |
| server/index.js                  | ~90            |
| <b>Total (TypeScript/TSX/JS)</b> | <b>~4,000+</b> |

Generated for portfolio and job application use. Keep in sync with major dependency or architecture changes.